Reinforcement Learning for Wrench-Limited Extraction

A detailed account of the PPO work behind the suction pick-and-place policy: the MDP and algorithm, the reward design, the full log of training-dynamics failures and their fixes, the speed-optimization experiments, and the evidence-based conclusion that on this task reinforcement learning is a diagnostic rather than the generator of the final policy.

Repository: `frankkimrobotics/pick_n_place` · Code: `rl/wrench_extract.py` (env + compact PPO), `rl/env_warp.py` (base twin) · Companion: `docs/report-wrench-extraction.md`

---

1. Formulation

1.1 MDP

- **State / observation** (33-D, policy input): joint angles and velocities (12), tool pose *relative to the array centre* (3), cup z-axis (3), object-relative offset (3), box-relative offset (2), height beliefs (3), wrench readout — pull, shear, peel, torsion, margin (5), sealed flag (1), normalized episode time (1). Observation noise σ = 0.005 at train and test.
- **Action** (7-D, in [−1, 1] via tanh): per-joint target deltas scaled by a 3.2°/decision clamp (6), and a suction logit (1). Suction release is *physically gated* by the executor (see §3.1), so the logit's effect is masked until the block is cleared and over the box.
- **Dynamics**: the batched MuJoCo-Warp twin with the measured Pro 630 velocity-mode drive and drive domain randomization. 10 Hz decisions, 50 physics substeps each.
- **Episode**: up to 165 decisions (home-start full task). Mixed start modes (§3.4). Terminates on place, drop-off-table, or timeout.

1.2 Algorithm

A compact, self-contained **PPO** (actor–critic, `wrench_extract.AC` / `train`):

- MLP policy and value heads, hidden width 256 → **512**; a learned state-independent `log_std`.
- **GAE** advantages, γ = 0.98, λ = 0.95.
- **Clipped surrogate**, ratio clip [0.8, 1.2]; value loss weight 0.5; entropy bonus (0.004 → 0.0005 over the project); gradient-norm clip 1.0; Adam.
- Batched rollouts: 512 parallel worlds × up to 32 variants, rollout length 32, several epochs per batch.

Two mechanisms were added after the failures below and are load-bearing:

- **KL early-stop**: abort the epoch loop once the approximate KL between old and new policy exceeds a threshold (0.02–0.06). Guards against the single destructive update.
- **Bounded `log_std`** (clamped to a ceiling) and a **score-gated best checkpoint** (a scalar that weights full-task success first, then speed), so the saved artifact is never a collapsed iterate.

---

2. Reward design

The reward is dense and potential-based where possible, with hard terminal events. Per decision (abbreviating potentials φ with their per-step increments):

TERM	FORM	PURPOSE
clearance progress	`50 · Δφ_clear · sealed`	lift the block until its bottom tops the true neighbours
extraction bonus	`+4` once, on first clear	a one-time reward so clearing is an event, not a rate
transport	`18 · Δφ_transport · (cleared & sealed)`	move toward the box, but only after clearing
descent	`4 · Δφ_descent · (cleared & sealed)`	lower toward the box floor when above it
height ceiling	`3 · Δφ_ceiling · sealed`	discourage carrying above 0.30 m (potential, costs once)
wrench prudence	`−0.05 · (1 − margin) · sealed`	stay inside the admissible set
neighbour disturbance	`−35 · max(0, ΔΣ‖shift‖)`	penalize nudging neighbours (increases only)
time	`−0.01`	mild efficiency
detach	`−6 · broke`	losing the seal is costly
place	`+25 (+ speed bonus)` on in-box landing	the goal
off-table	`−5`	drop penalty

Several of these shapes are the *result* of a failure (below), not a priori choices: the one-time extraction bonus, the `cleared &` gates on transport/descent, and the box-gated release all exist to close specific reinforcement-learning exploits.

Potentials are reset correctly at episode start (a subtle but real bug source — a potential that carries across a reset injects a spurious reward spike).

---

3. Training dynamics: the failure log

The extraction policy took many PPO iterations. Each plateau or collapse had a specific, diagnosable cause; the fixes compound into the reward and gating design above.

3.1 The "give up instantly" optimum

With a *free* suction logit, episodes collapsed to ~15 decisions: the policy released immediately to end the episode and avoid accumulating any risk. A −6 penalty for premature release did **not** fix it — the fresh policy's logit flips randomly and the penalty is diffuse. The fix was **structural**: physically gate release until the block is cleared, then (a second iteration) until it is cleared *and over the box*. With no cheap exit, the only terminations are detach, timeout, or place — and the transport shaping finally gets full-length episodes to work on.

3.2 The 540k collapse

A run would climb to ~64 % clearance by ~500k steps, then slide to 14 % and not recover — a catastrophic, unrecoverable drift out of the learned basin. Fixes: **KL early-stop** (one update was walking the policy far off-distribution), a **`log_std` ceiling** (free std was inflating), a lower learning rate, and a **score-gated best checkpoint** so the 64 % policy is banked regardless of what happens afterward. (Gating the checkpoint on raw success failed because success is 0 for a long time, so every early iterate — including collapsed ones — looked equally good.)

3.3 Entropy-driven decay

Even with those guards, clearance would peak (~64 %) and then slowly erode with rising break rates. The `log_std` was pinned at its ceiling: the entropy bonus kept inflating action noise as advantages flattened, and noisy joint deltas are exactly what violate the wrench set. Cutting the entropy coefficient and tightening the std ceiling stabilized it.

3.4 The carry was never discovered — mix curriculum

A single network trained from home could clear the block (89 %) but **never placed it** (0 %): a probe showed it extracting and then climbing vertically to z ≈ 0.69 until timeout — it had no signal for the carry. Transport shaping at 8× did not help; raising it to 18× plus a height ceiling did partially. The decisive fix was a **mix curriculum**: 40 % of episodes start from home, but 20 % start sealed in the array, 15 % mid-carry, and 25 % already over the box. Value learned from the easy over-box placements propagates back through the shared network to teach the carry. Per-mode success is logged separately so the home-start (full-task) number stays honest.

3.5 The plateau, and the pivot

After all of the above, single-network PPO plateaued at roughly **30 % full-task success**, flat over long runs. The carry and place existed in the network (the curriculum had taught them in isolation) but the policy could not reliably *stitch* extraction → carry → place in a single home-start episode. At this point the physics bugs (see the companion report) were found by tracing a scripted expert, which immediately reached 128/128. The project pivoted to **behaviour cloning from that expert**; PPO had done its job of validating the design and exposing the physics.

---

4. Speed-optimization experiments

A later question — *can the finished policy go faster without violating the constraint?* — was a second, self-contained RL study on top of the 12/12 behaviour clone. It is the cleanest illustration of reward-shaping pitfalls in the whole project.

4.1 Over-weighted time penalty → bailing

First attempt: raise the per-step time cost 4× (−0.01 → −0.04) and add a +12 early-finish bonus. Result: success and clearance **collapsed** (100 % → 67 %), cycle barely moved. The −0.04/step accumulates to ~−6.6 over an episode, rivaling the +25 placement — so *giving up* became competitively rewarding. A time penalty that can outweigh success teaches the policy to fail faster, not succeed faster.

4.2 Success-conditional speed bonus

Fix: never pay for time directly. Keep a mild −0.01/step and make speed reachable **only through a placement-conditional bonus** (+30 scaled by time remaining). Now a successful episode is worth +25 to +55 and a failure is 0 — failing is always strictly worse than succeeding slowly, and the bail exploit is closed by construction. This trained cleanly: 99–100 % success, 0 breaks, and cycle dipping (apparent 92 at one checkpoint).

4.3 The exploration-noise trap, and the recurring ~130k collapse

Two further findings:

- Training-time action sampling at σ = 0.30 is ~2.5× the DART noise the press was distilled to tolerate; it wrecked the seal on home starts (home success 0 % while sealed-start modes were 100 %). Capping σ at 0.20 restored it.
- Both speed runs **collapsed at ~130k steps** — the same step in two runs with different reward shaping — indicating policy drift out of the clone basin after a few update batches, despite the KL cap.

4.4 Verdict: the best PPO policy equals the clone

Evaluated deterministically on 12 unseen scenes, the score-gated best speed-round checkpoint is **identical to the clone**: 12/12, 0 breaks, 106 decisions. The score gate correctly preferred the first 100 %-success iterate (weighting success ≫ speed), and the apparent speedups always came bundled with a success dip. Separately, amplifying the clone's actions at inference by 1.0/1.3/1.6× also gave an identical 106 decisions — the policy uses only ~56 % of its action clamp in free motion, so the cycle is not speed-limited there. The remaining time is in the seal-limited press and carry, where the margin *is* the safety buffer. **PPO cannot buy speed there without spending the constraint.**

---

5. What RL achieved, and the comparison

APPROACH	FULL-TASK RESULT	ROLE
PPO from scratch	plateaued ~30 % home success	validated reward/gating; **found the physics bugs**
Behaviour cloning (scripted expert)	**12/12** unseen, 0 breaks	the deployed policy
DAgger (on the clone)	9/12 → 0/12 (regressed, twice)	fails: stateful expert mislabels off-trajectory states
PPO fine-tune (from the clone)	overfit (4/12) or collapsed (~130k)	no improvement over the clone

The honest summary: reinforcement learning was indispensable as the **instrument** that proved the task solvable and surfaced four simulator physics defects that no amount of reward tuning would have fixed — but it never produced weights that beat a clone of the corrected scripted expert.

---

6. Lessons for RL on this stack

1. **Instrument the physics before tuning the reward.** ~7 PPO runs were spent against blockers that were contact bugs, not learning problems. A scripted expert + a force trace found them in under an hour.
2. **Close exploits with structure, not penalties.** The "give up instantly" and "drop early" optima were defeated by *physically gating* release, not by penalty terms that a fresh policy's noisy logit ignores.
3. **Never let a per-step cost outweigh the terminal success.** Shape for speed only *conditional on completion*; an unconditional time penalty teaches faster failure.
4. **Guard the policy from itself.** KL early-stop, a `log_std` ceiling, a cut entropy bonus, and a score-gated best checkpoint were each necessary to keep a good iterate from being overwritten by a collapse.
5. **A long-horizon multi-phase skill may need a start-state curriculum**, not just dense shaping — value from easy terminal states teaches the phases exploration from the initial state never reaches.
6. **Match exploration noise to what downstream training tolerated.** σ above the distillation noise floor silently destroyed the most delicate sub-skill (the seal) while the rest looked healthy.

7. **Know when RL has done its job.** Here its value was diagnostic; recognizing the ~30 % plateau as a stitching failure (not a tuning problem) and pivoting to imitation was the decision that produced the result.

Generated as part of the pick_n_place program.